HUMAN ACTIVITY RECOGNITION FROM SMARTPHONE MOTION SENSORS USING A CONV1D–BILSTM–ATTENTION MODEL

PROJECT REPORT [Phase-I]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

Submitted by

Devansh Shankhadhar (24U020027)

Hemant Nagar (24U020015)

Under the supervision of

Dr. Yatendra Sahu

Assistant Professor

Dr. Ridhi Arora

Assistant Professor

Computer Science and Engineering

November 2026

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



CERTIFICATE

This is to certify that the work embodied in this report entitled **"HUMAN ACTIVITY RECOGNITION FROM SMARTPHONE MOTION SENSORS USING A CONV1D–BILSTM–ATTENTION MODEL"** has been satisfactorily completed by **Devansh Shankhadhar (24U020027)** and **Hemant Nagar (24U020015)**. It is a bonafide piece of work, carried out under our guidance in the **Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal** for the partial fulfillment of the Bachelor of Technology during the academic year 2026-2027.

Date: _____

Assistant Professor,	**Assistant Professor,**	**Department Coordinator,**
Dr. Yatendra Sahu,	Dr. Ridhi Arora,	Dr. Rahul Chaurasia
Computer Science and Engineering,	Computer Science and Engineering,	Department of CSE,
IIIT Bhopal (M.P.)	IIIT Bhopal (M.P.)	IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)



DECLARATION

We hereby declare that the following project report entitled **"HUMAN ACTIVITY RECOGNITION FROM SMARTPHONE MOTION SENSORS USING A CONV1D–BILSTM–ATTENTION MODEL"** is presented in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering. It is an authentic documentation of our work carried out under the able guidance of Dr. Yatendra Sahu and Dr. Ridhi Arora. The work has been carried out at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole for the award of any degree or professional diploma in any other institute or organization.

We hereby declare that the facts mentioned above are true to the best of our knowledge. In case of any discrepancy that may occur, we will be the ones to take responsibility.

Devansh Shankhadhar (24U020027) Signature: _____

Hemant Nagar (24U020015) Signature: _____

ACKNOWLEDGEMENT

We, Devansh Shankhadhar and Hemant Nagar, would like to express our sincere gratitude to everyone who supported and guided us throughout the completion of our project titled **“Human Activity Recognition from Smartphone Motion Sensors Using a Conv1D–BiLSTM–Attention Model”**.

First and foremost, we would like to thank our Director, **Prof. Dr. Ashutosh Kumar Singh**, for providing the resources and facilities needed to carry out this project.

We are deeply grateful to our Project Guides, **Dr. Yatendra Sahu** and **Dr. Ridhi Arora**, Assistant Professors, Department of Computer Science and Engineering, for regular guidance and for the suggestions that shaped the evaluation of the model.

Our sincere thanks to the Department Coordinator, **Dr. Rahul Chaurasia**, for the support and encouragement provided during the project, and to the faculty of the Department of Computer Science and Engineering for their teaching and support.

We also thank our parents and families for their patience and moral support.

Devansh Shankhadhar
24U020027
Hemant Nagar
24U020015

AREA OF WORK

Our project focuses on **Human Activity Recognition (HAR)** using **deep learning for time-series classification**. The aim is to decide automatically, from the accelerometer and gyroscope of an ordinary smartphone, which of six activities a person is performing.

The work covers signal processing of accelerometer and gyroscope data, the combination of several public datasets into one consistent training set, a hybrid convolutional and recurrent network with temporal attention, and its evaluation on people and recordings not used for training, with attention to data leakage, units and sampling rates. The trained model is served by a Flask REST API to a browser client that reads a phone's motion sensors, with predictions stored in SQLite.

TABLE OF CONTENTS

S.No.	Title	Page No.
	Abstract	1
1	Introduction	2
2	Background Details and Literature Review	4
3	Problem Definition and Objectives	8
4	Proposed Methodology and Work Description	9
5	Proposed Algorithms	17
6	Proposed Flowcharts and Diagrams	19
7	Implementation (Tools and Technology Used)	21
8	Result Discussion and Analysis	23
9	Conclusion and Future Scope	28
	References	30
	List of Publications	32
	Plagiarism Report	33

LIST OF FIGURES

Fig. No.	Description	Page No.
1.	Data Preparation and Training Pipeline	11
2.	Conv1D–BiLSTM–Attention Model Architecture	15
3.	High-Level System Architecture	19
4.	Prediction Pipeline Flowchart	20
5.	Confusion Matrix on Held-Out Public-Dataset Subjects (percentage of each true class)	24

LIST OF TABLES

Table No.	Description	Page No.
1.	Comparison of HAR Approaches in Literature	7
2.	Data Sources Used for Training and Testing	10
3.	Activity Class Mapping	12
4.	Division of the Data by Person and Recording (windows before capping and augmentation)	12
5.	Tools and Technologies Used	21
6.	Test Sets Used in the Evaluation	23
7.	Per-Class and Overall Results on Held-Out Public-Dataset Subjects (model output)	24
8.	Results per Source Dataset (held-out subjects, model output)	25
9.	Results on Phone Recordings (accuracy of model output / of dashboard pipeline)	26
10.	Comparison with Published Approaches	27

ABSTRACT

Human Activity Recognition (HAR) is the task of identifying what a person is doing, such as walking or climbing stairs, from sensor data. This project builds a HAR system that uses the accelerometer and gyroscope of a smartphone. A web page on the phone samples the sensors at 20 Hz and sends a three-second window of six signals to a Flask server every 1.5 seconds. The server filters the window, removes gravity, adds two magnitude channels, and classifies the result with a neural network made of one-dimensional convolutional layers, two bidirectional Long Short-Term Memory (BiLSTM) layers and a temporal attention layer. The six output classes are Hand Activity, Jogging, Sports, Stairs, Still and Walking. Rule-based post-processing and per-session smoothing stabilise the label, and every prediction is stored in an SQLite database with CSV and PDF export.

The model is trained on three public datasets (WISDM, UCI HAR and Heterogeneity) and a small set of phone recordings. Every source is first converted to the units, sampling rate and window length of the live input, and the data are split by person and by recording before augmentation, so that no test subject or recording contributes to training. On 27,352 windows from public-dataset subjects excluded from training, the model reaches an accuracy of 77.9 percent, a macro precision of 0.807, a macro recall of 0.798 and a macro F1-score of 0.796; Stairs recall is 0.928, while Hand Activity, which is often confused with Still, is the weakest class (recall 0.473). On the only held-out phone stair-climbing session, 54 of 117 windows are classified as Stairs. This session is the only labelled phone test data for Stairs, so the result does not establish performance on phones in general, and the accuracy target of 80 percent was not reached.

Keywords: Human Activity Recognition, smartphone sensors, Conv1D, Bidirectional LSTM, temporal attention, data leakage, Flask, React

INTRODUCTION

1.1 Background

Almost every smartphone contains a three-axis accelerometer, which measures linear acceleration, and a three-axis gyroscope, which measures the rate of rotation. Walking, running or climbing stairs with a phone in a pocket or in the hand produces characteristic patterns in these signals, and Human Activity Recognition (HAR) uses such patterns to identify the activity automatically, with applications in fitness tracking, health monitoring and context-aware services [1], [2]. Early HAR systems computed hand-designed features from short windows and classified them with methods such as Support Vector Machines or decision trees [1], [2]. Deep neural networks instead learn features directly from the signal: convolutional layers detect short local patterns, recurrent layers such as the Long Short-Term Memory (LSTM) model their temporal order, and hybrid networks combine both [3], [4], [5], [6].

1.2 Motivation

Most published HAR results are obtained offline on a single benchmark dataset, with one device, one body position and a fixed protocol. A phone-browser application does not share these conditions. Its input comes from the DeviceMotion interface of a mobile browser, sampled at 20 Hz in m/s² with gravity included, from a phone held in the hand or carried in a pocket. No public dataset was recorded in exactly this format, so a model can only be trained by combining datasets whose units, sampling rates and labels differ. Every mismatch between training data and live input lowers recognition quality, and even differences between devices alone measurably reduce performance [7].

Two activities make this problem concrete. Walking and stair climbing produce similar periodic acceleration patterns and are easily confused [2], and hand-based activities produce little motion at a phone in the pocket. A useful system must therefore (i) convert all training sources to the live-input format, (ii) evaluate on people and recordings that were never used for training, because overlapping windows and augmented copies inflate accuracy when they cross the train–test boundary (Section 2.7), and (iii) turn per-window predictions into a stable, stored record of activity. This project addresses these three requirements with a Conv1D–BiLSTM–attention model served by a Flask API to a browser dashboard.

1.3 Scope of the Project

1. A Conv1D–BiLSTM–attention model with six output classes (Hand Activity, Jogging, Sports, Stairs, Still and Walking), trained on the WISDM [8], UCI HAR [1] and Heterogeneity [9] datasets and a few phone recordings.
2. A data-preparation pipeline that converts all sources to the live-input format and splits the data by person and by recording before augmentation to avoid the evaluation bias described in [10], and an evaluation of the model on held-out subjects and phone recordings, supported by reproducible evaluation scripts and 25 automated tests.
3. A Flask REST API that preprocesses each window with the same function used for training, runs the model, applies post-processing and per-session smoothing, similar

in purpose to the filtering of consecutive outputs in real-time smartphone HAR [11], and stores the predictions in SQLite with CSV and PDF export.

4. A lightweight browser client (React and Vite) that streams 20 Hz sensor windows, the sampling rate also used by the WISDM dataset [8], to the API and displays the predicted activity. It serves as the test and demonstration front end of the model rather than as a research contribution. Google sign-in, MongoDB logging and Gemini-generated reports exist in the code as an optional module that is disabled without credentials; it was not tested and is outside the scope of this report.

1.4 Organisation of the Report

Chapter 2 reviews the relevant datasets [1], [8], [9] and deep-learning methods [4], [6], [12], and Chapter 3 defines the problem and objectives. Chapter 4 describes the data pipeline, model, training and real-time prediction; Chapter 5 gives the algorithms and Chapter 6 the main diagrams. Chapter 7 summarises the implementation, Chapter 8 presents the results and limitations, and Chapter 9 concludes the report.

BACKGROUND DETAILS AND LITERATURE REVIEW

2.1 Introduction

A typical HAR system divides the continuous sensor signal into short, fixed-length windows and assigns one activity label to each window. Early systems passed hand-designed window features to classical classifiers [1], [2], while later research showed that deep networks can learn features directly from raw or lightly processed signals [3], [4], [5]. Sections 2.2 to 2.4 review the three public datasets used in this project and how their labels relate to our six classes, Sections 2.5 and 2.6 review the relevant deep-learning methods, and Section 2.7 summarises the research gaps.

2.2 Research on the UCI HAR Dataset

Anguita et al. introduced the UCI Human Activity Recognition dataset in 2013 [1]. Thirty volunteers performed six activities, namely walking, walking upstairs, walking downstairs, sitting, standing and lying, while a smartphone was fixed to their waist. The accelerometer and gyroscope signals were divided into windows, and 561 time-domain and frequency-domain features were calculated from each window. A multiclass Support Vector Machine (SVM) trained on these features reached an overall accuracy of 96 percent on 2,947 test windows [1]. The dataset is distributed by the UCI Machine Learning Repository as "Human Activity Recognition Using Smartphones".

Official dataset page [1]: **https://archive.ics.uci.edu/dataset/240/**

Reyes-Ortiz et al. extended this work to transitions between activities, such as standing up or lying down, with a real-time system that combines consecutive SVM outputs with heuristic filtering [11]. The dataset became a common benchmark because of its fixed division into training and test subjects [13], [14], [15], although its strong results were obtained with hand-designed features, a single body position and a controlled recording procedure.

In our project, the raw inertial signals of this dataset [1] are part of the training data, mapped to Walking, Stairs and Still (Table 3). They differ from our phone input: they are sampled at 50 Hz in units of g from a phone worn on the waist, whereas our input is sampled at 20 Hz in m/s^2 with gravity included from a phone held in the hand or kept in a pocket. The pipeline therefore uses the total-acceleration signals, converted to m/s^2 and resampled to 20 Hz (Section 4.3).

2.3 Research on the WISDM Dataset

The Wireless Sensor Data Mining (WISDM) laboratory at Fordham University has produced two well-known activity datasets. In the earlier study, Kwapisz et al. collected accelerometer data from 29 users who carried a phone in their pocket while walking, jogging, climbing up and down stairs, sitting and standing [2]. They calculated 43 features from 10-second segments and compared decision trees, logistic regression and a multilayer perceptron. The multilayer perceptron achieved the highest overall accuracy of 91.7 percent, and climbing up and down stairs were the hardest activities to recognise [2]. This earlier dataset contains accelerometer data only and is not the WISDM dataset used in our project.

In 2019, Weiss et al. released the WISDM Smartphone and Smartwatch Activity and Biometrics Dataset [8], [16]. It contains accelerometer and gyroscope data from a smartphone and a smartwatch, recorded at 20 Hz from 51 subjects who each performed 18 activities for 3 minutes [8]. The activities include basic movements such as walking, jogging and climbing stairs, hand-based activities such as brushing teeth and folding clothes, and several eating activities. The phone was kept in the subject's pocket and the watch was worn on the dominant hand [8], [17]. The main aim of the original study was biometrics: Weiss et al. showed that a person can be identified from the way they perform everyday activities [16]. The dataset is available from the UCI Machine Learning Repository.

Official dataset page [8]: **https://archive.ics.uci.edu/dataset/507/**

Oluwalade et al. used the same dataset for activity recognition [17]. They found a statistically significant difference between the data recorded by the phone and by the watch and explained it by the different places where the two devices are worn. After combining the eating activities into one class, which left 15 activities, they trained LSTM, bidirectional LSTM, CNN and convolutional LSTM models on watch accelerometer data. Activities not performed with the hands, such as standing and stairs, were generally classified better than hand-oriented activities [17]. Alam et al., who used the dataset in a federated-learning benchmark, note that the phone and watch data were not recorded at the same time for most subjects [18].

Our project uses only the smartphone accelerometer and gyroscope files of this dataset [8]; the mapping of its activity codes to our classes is given in Table 3, and the eating and drinking activities are not used. WISDM is our only source of Hand Activity and Jogging data and, together with the "bike" activity of the Heterogeneity dataset, of Sports data. Hand Activity describes hand-based tasks recorded while the phone was in the pocket, so the phone sees only the small body movement that accompanies the hand movement; because hand-oriented activities were harder to classify even with a watch on the moving hand [17], this class was expected to be difficult (Chapter 8).

2.4 Research on the Heterogeneity Activity Recognition Dataset

Stisen et al. studied how differences between devices affect activity recognition [7]. They collected data with 36 smartphones and smartwatches of 13 different models from four manufacturers, used by nine people, and showed that differences in sensor hardware and in the way each device handles its sensors measurably reduce recognition performance [7]. The data were released as the Heterogeneity Activity Recognition dataset [9], with accelerometer and gyroscope readings for biking, sitting, standing, walking, stair up and stair down, plus a null class. Each device was sampled at the highest frequency it allowed, and the documentation does not state where the phones were carried [9].

Official dataset page [9]: **https://archive.ics.uci.edu/dataset/344/**

Following the label names of the dataset [9], its activities are mapped to Walking, Still, Stairs and Sports (Table 3), and each device's readings are aligned by timestamp and resampled to 20 Hz (Section 4.3). Our live input comes from yet another source, the motion sensors exposed by a mobile web browser; device differences are known to reduce recognition performance [7], but the effect of this particular difference has not been measured.

2.5 Convolutional, Recurrent and Attention Models

Convolutional networks. LeCun et al. showed that networks built from local receptive fields, shared weights and sub-sampling learn features directly from data instead of relying on hand-designed features [19]. Applied in one dimension along the time axis, a convolutional filter responds to short sensor patterns such as the peak of a footstep. Zeng et al. were among the first to apply CNNs to mobile-sensor activity recognition and outperformed the methods of the time on the Skoda, Opportunity and Actitracker datasets [4]; Yang et al. learned features from raw multichannel signals jointly with the classifier [3], and Ha and Choi combined several accelerometers and gyroscopes in one network [20]. Because a convolutional layer sees only a limited stretch of the signal, hybrid models add recurrent layers after it [6].

Recurrent networks. The LSTM of Hochreiter and Schmidhuber uses gated memory cells to keep information over long sequences [21], and the bidirectional network of Schuster and Paliwal reads a sequence forwards and backwards so that each output uses both earlier and later context [22]; in window-based HAR the whole window is available, so this needs no future data. Hammerla et al. compared fully connected, convolutional and recurrent models on three wearable datasets and gave guidelines for their use [5]. Guan and Plötz improved on single LSTMs with LSTM ensembles [23], Zhao et al. proposed a residual bidirectional LSTM with reported gains on Opportunity and UCI HAR [13], and Chen et al. reported 92.1 percent accuracy with an LSTM on WISDM accelerometer data [24].

Temporal attention. Bahdanau et al. replaced the fixed-length encoding of a sequence with learned soft weights over its parts [25], and Murahari and Plötz introduced attention layers into DeepConvLSTM for wearable HAR, reporting improved performance and interpretable temporal weights [12]. The attention layer of our model (Section 4.7) follows this general idea of learned soft weights over time steps, but it is not the same design as [25] or [12].

2.6 Hybrid Models and Data Augmentation

Ordóñez and Roggen proposed DeepConvLSTM, with four convolutional layers followed by two LSTM layers [6]. On the Opportunity dataset it outperformed deep non-recurrent networks by 4 percent on average and reached a weighted F1-score of 0.915 for gesture recognition including the Null class [6]. Mutegeki and Han reported 92 percent accuracy on UCI HAR with a CNN-LSTM model [14], and Xia et al. placed two LSTM layers before the convolutional layers and reported 95.78 percent on UCI HAR, 95.85 percent on WISDM and 92.63 percent on Opportunity [15]. Um et al. compared augmentation methods for wearable sensor data and found rotation to be the best single method; combining rotation, permutation and time warping raised accuracy from 77.54 to 86.88 percent [26]. Our model belongs to this hybrid family and our pipeline uses rotation-based augmentation (Section 4.5). The quoted figures come from different datasets and protocols and are not directly comparable.

2.7 Research Gaps and Relevance to Our Project

The way a model is evaluated strongly affects the reported accuracy. Hammerla and Plötz showed that random cross-validation on segmented time-series data is biased, because neighbouring windows are not statistically independent, and that performance drops when this bias is removed [10]. With 50 percent window overlap and augmented copies, a random

window-level split would place near-duplicates of the same recording in both the training and the test data. The pipeline of this project therefore splits the data by person and by recording before augmentation (Section 4.5).

A second gap is generalisation across devices and body positions. Stisen et al. showed that differences between devices reduce recognition performance [7], and Oluwalade et al. found that a phone and a watch record the same activities differently because of where they are worn [17]. The datasets used in our project were recorded with a phone on the waist [1], in the pocket [8], or with many different devices [7], while our users may hold the phone in the hand or keep it in a pocket.

Most of the reviewed deep-learning studies report results on recorded benchmark datasets [3], [5], [13], [6], [15]; real-time phone-based recognition has been addressed [11], but less attention is given to the full path from live browser sensor data to a stored record of activity. This project therefore contributes a consistent data pipeline, a working system and a leakage-free evaluation rather than a more accurate model, and its measured results do not exceed the published methods. Table 1 summarises the reviewed studies that are most relevant to the methodology of this project.

Table 1: Comparison of HAR Approaches in Literature

Approach	**Representative Work**	**Main Contribution**	**Relevance to This Project**
Features with classical classifiers	Kwapisz et al. [2]	Pocket phone accelerometer; 91.7% with a multilayer perceptron	Earlier WISDM study; stairs were hardest to recognise
Device heterogeneity	Stisen et al. [7]	Device differences reduce recognition performance	Heterogeneity dataset is part of our training data
1-D CNN	Zeng et al. [4], Yang et al. [3]	Features learned from raw sensor signals	Basis of our convolutional blocks
LSTM and BiLSTM	Hammerla et al. [5], Zhao et al. [13]	Modelling of temporal order in sensor windows	Basis of our BiLSTM layers
CNN-LSTM hybrid	Ordóñez and Roggen [6], Xia et al. [15]	Local features combined with temporal modelling	Same family as our model
Temporal attention	Murahari and Plötz [12]	Learned weights over temporal context	Basis of our attention layer
Data augmentation	Um et al. [26]	Rotation was the most useful single augmentation	Rotation is used in our training pipeline
Evaluation bias	Hammerla and Plötz [10]	Random splits of overlapping windows inflate results	Reason for our person-level split before augmentation

PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem Statement

The problem addressed in this project is to recognise six physical activities (Hand Activity, Jogging, Sports, Stairs, Still and Walking) in real time from the accelerometer and gyroscope of an ordinary smartphone, using only the phone's web browser as the sensing client, and to measure the recognition performance on people and recordings that were not used for training.

The problem has three technical parts. First, the training data come from three public datasets and a few phone recordings that differ in units, sampling rate, body position and activity labels, whereas the live input has a fixed format (m/s² including gravity, rad/s, 20 Hz, 3-second windows); all sources must be brought into this format. Second, the evaluation must be free of leakage between overlapping windows and augmented copies, which requires a split by person and by recording made before augmentation. Third, per-window predictions must be stabilised and stored so that a user can read and review the recognised activity.

3.2 Objectives

1. To design a six-class Conv1D–BiLSTM–temporal-attention classifier for smartphone accelerometer and gyroscope windows, with a target accuracy of at least 80 percent on subjects excluded from training.
2. To build a data-preparation pipeline that converts the WISDM, UCI HAR and Heterogeneity datasets and the project's phone recordings to the units, sampling rate and window format of the live input.
3. To evaluate the model without data leakage, using person- and recording-level splits made before augmentation, and to report per-class and overall metrics.
4. To implement a real-time inference service that shares its preprocessing with training and stabilises predictions with rule-based post-processing and per-session smoothing.
5. To develop a browser dashboard for live sensing, CSV analysis and replay, with SQLite session storage and CSV and PDF export.

Section 9.2 reviews which of these objectives were met.

PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 System Overview

The system has a client-server structure with four parts: (a) a web dashboard that runs in the phone's browser and reads the motion sensors, (b) a Flask server that preprocesses the data and runs the model, (c) a local SQLite database that stores recording sessions and predictions, and (d) an optional cloud module for sign-in and AI-generated reports, which is switched off unless credentials are configured.

The browser samples the accelerometer (m/s^2, including gravity) and the gyroscope (converted from degrees per second to radians per second) every 50 ms, that is at 20 Hz. After the first 60 samples (3 seconds) it sends the most recent 60 samples to the server, and then repeats this every 30 new samples (1.5 seconds), so consecutive windows overlap by 50 percent. The server converts the 60×6 window into a 60×8 feature window with the same function that was used to build the training data, normalises it, runs the neural network, applies post-processing rules and per-session smoothing, stores the prediction, and returns it to the dashboard.

The model is trained offline with the pipeline described in Sections 4.2 to 4.8. The trained network, its fitted scaler and its label encoder are stored together in one model folder, which the server loads and checks at start-up (Section 7.3). Sessions and predictions are stored in a local SQLite database (backend/data/har.db) that needs no installation or credentials. The optional cloud module (backend/cloud_reports.py) adds Google sign-in, MongoDB logging and Gemini-generated daily reports when credentials are configured; no credentials were available, so it was not tested and is not part of the evaluated system.

4.2 Data Sources

The training and test data come from three public datasets and the project's own phone recordings (Table 2). All four sources contain three-axis accelerometer and three-axis gyroscope readings from a smartphone.

Table 2: Data Sources Used for Training and Testing

Source	People	Phone position	Original rate and units	Windows
WISDM 2019 [8], [16]	51	Pocket	20 Hz; m/s² with gravity; rad/s	77,484
Heterogeneity [7], [9]	9	Not stated	Highest rate of each device; units not stated (Android default of m/s² and rad/s assumed)	44,014
UCI HAR [1]	30	Waist	50 Hz; acceleration in g; rad/s	8,186
Project phone recordings	Not documented	Not documented	Varies by file (for example 44–52 Hz accelerometer and 15–22 Hz gyroscope in the Stairs session); m/s² and rad/s	395
Total				**130,079**

The phone recordings were made with an Android sensor-logging application (rows of Timestamp, Sensor Type, Value1–3; type 1 is the accelerometer and type 4 the gyroscope). Six labelled recordings are used: one Still, one Jogging, two Walking and two Stairs files. Their timestamps show that the two Stairs files cover the same 95 seconds at different sampling rates, so they are one stair-climbing session recorded on two devices. Three further files are byte-identical copies of other recordings and are excluded.

4.3 Converting All Sources to a Common Format

The live application always sends acceleration in m/s² including gravity, angular velocity in rad/s, and samples spaced exactly 50 ms apart. The pipeline therefore converts every source into this format before any window is cut:

- **UCI HAR.** The total-acceleration signals (in g, with gravity) are multiplied by 9.80665 to obtain m/s². The dataset is distributed as 2.56-second windows with 50 percent overlap; consecutive windows of the same person and activity are joined back into continuous 50 Hz signals where their overlapping halves match, and these signals are resampled to 20 Hz with a polyphase filter that includes anti-aliasing.
- **WISDM.** The timestamps are converted from nanoseconds to milliseconds, and the accelerometer and gyroscope readings of each subject and activity are aligned by time and interpolated onto a common 20 Hz grid.
- **Heterogeneity.** The readings of each user, device and activity are aligned by time in the same way; the null class is dropped.
- **Phone recordings.** The files are read with parse_sensor_csv, the same function that the server uses for uploaded CSV files.

For all sources, the alignment step splits the data wherever either sensor has a gap of more than one second, so gaps are never filled by interpolation. Each continuous segment is then cut into windows of 60 samples (3 seconds) with a step of 30 samples. The windows are

stored with their activity label, source, person (or recording) and split in one file. Figure 1 shows the complete data preparation and training pipeline.

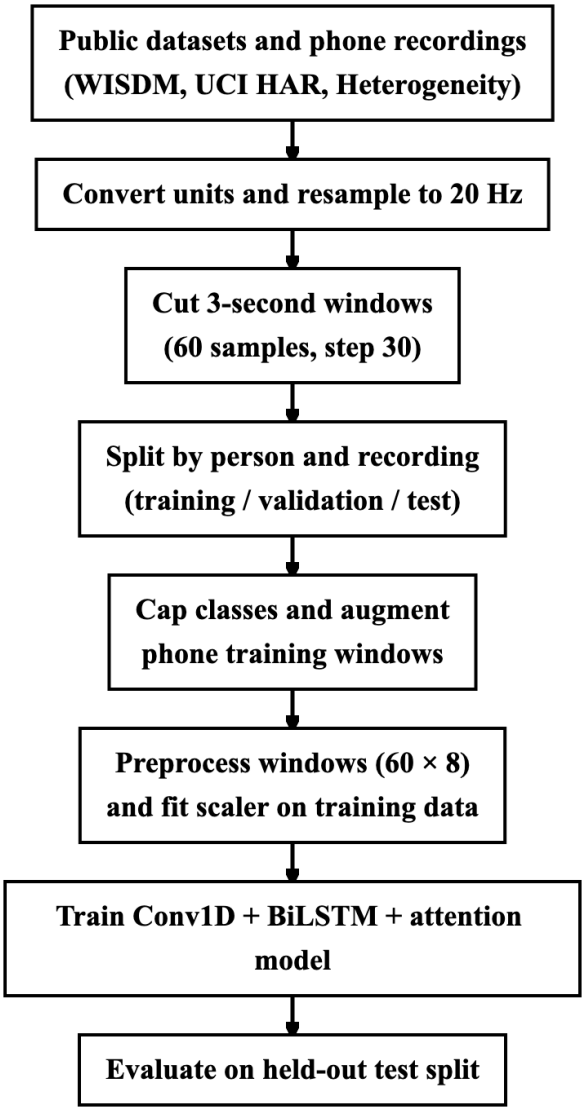


Figure 1: Data Preparation and Training Pipeline

4.4 Activity Class Mapping

The activity lists of the sources are merged into six classes (Table 3). Activities outside these classes, such as the eating and drinking activities of WISDM (codes H to L) and the null class of the Heterogeneity dataset, are not used, so the model cannot recognise them.

Table 3: Activity Class Mapping

Class	WISDM codes	Heterogeneity labels	UCI HAR labels	Phone recordings
Walking	A (walking)	walk	1 (walking)	2 files
Jogging	B (jogging)	—	—	1 file
Stairs	C (stairs)	stairsup, stairsdown	2, 3 (upstairs, downstairs)	2 files (one session)
Still	D (sitting), E (standing)	sit, stand	4, 5, 6 (sitting, standing, lying)	1 file
Hand Activity	F (typing), G (teeth), Q (writing), R (clapping), S (folding)	—	—	—
Sports	M (kicking), O (catch), P (dribbling)	bike	—	—

Stairs combines climbing up and climbing down, so the model cannot tell the two directions apart. Hand Activity and Sports have no phone recordings, so they cannot be tested on the project's own phone data.

4.5 Data Splitting, Augmentation and Class Balance

Splitting by person. The data are divided before any augmentation, and no person appears in more than one split (Table 4). For WISDM, ten subjects form the test set and five the validation set; for the Heterogeneity dataset, two users and one user; for UCI HAR, the nine official test subjects form the test set and three of the official training subjects the validation set. The subjects were chosen by a seeded random shuffle (seed 42) before any results were seen. Among the phone recordings, the stair-climbing session (both files) and one Walking recording are kept for testing, and the Still, Jogging and the other Walking recording are used for training. No phone Stairs data is therefore used for training.

Table 4: Division of the Data by Person and Recording (windows before capping and augmentation)

Source	Training	Validation	Test
WISDM	36 people, 54,575 windows	5 people, 7,552	10 people, 15,357
Heterogeneity	6 people, 29,524	1 person, 4,834	2 people, 9,656
UCI HAR	18 people, 4,905	3 people, 942	9 people, 2,339
Phone recordings	3 recordings, 230	—	3 recordings, 165
Total	**89,234**	**13,328**	**27,517**

Capping. To keep the training time on a laptop CPU reasonable, at most 2,500 training windows per class are taken from each public dataset, sampled across all training subjects, and at most 600 validation windows per class and dataset. Phone windows are never capped.

Augmentation. Only the training part is augmented, and only the few phone recordings, each window receiving ten augmented copies. Each copy applies changes that a real phone could produce: a random three-dimensional rotation applied to the accelerometer and gyroscope together (as if the phone were turned in the pocket), a change of amplitude of up to 10 percent, and small sensor noise (standard deviation 0.05 m/s² and 0.01 rad/s). Rotation was the most useful single augmentation in the study of Um et al. [26]. Augmentation is applied to the raw six-channel signal, before the magnitude channels are calculated, so the magnitudes cannot become negative.

Class balance. After capping and augmentation the training set contains 32,167 windows (Still 8,633, Walking 6,510, Stairs 6,320, Sports 5,000, Jogging 3,204 and Hand Activity 2,500), and the validation set 6,912 windows. The remaining imbalance is handled by balanced class weights, which give each class the same total weight in the loss: Hand Activity 2.14, Jogging 1.67, Sports 1.07, Stairs 0.85, Walking 0.82 and Still 0.62.

4.6 Signal Preprocessing

Every 60 × 6 window, from the training data, an uploaded CSV file or the live phone, is converted into a 60 × 8 feature window by the same function (preprocess_window in backend/preprocessing.py; Algorithm 2), so the model always receives input processed exactly like its training data.

1. **Noise reduction.** A third-order Butterworth low-pass filter with a cut-off frequency of 5 Hz is applied to each of the six channels. Human movements such as steps lie mostly below this frequency. The filter is applied forwards and backwards (zero-phase filtering with scipy.signal.sosfiltfilt), so it does not shift the signal in time.
2. **Gravity separation.** A second Butterworth low-pass filter with a cut-off of 0.3 Hz extracts the slowly changing gravity component from the filtered accelerometer channels. Subtracting it gives the body acceleration, which depends on the movement rather than on how the phone is held.
3. **Magnitude features.** Two channels are added: the magnitude of the body acceleration, $\sqrt{(bx^2 + by^2 + bz^2)}$, and the magnitude of the angular velocity, $\sqrt{(gx^2 + gy^2 + gz^2)}$. Magnitudes do not change when the phone rotates in the pocket.
4. **Normalisation.** Each of the eight channels is standardised with a StandardScaler (subtracting the mean and dividing by the standard deviation). The scaler is fitted on the training windows only, and the fitted scaler is saved with the model (scaler.pkl) and reused at inference time.

The output channels are body acceleration x, y, z, angular velocity x, y, z and the two magnitudes. Filtering is done separately for each window and no filter state is kept between requests, so different sessions cannot influence each other.

4.7 Model Architecture

The network (create_model in backend/train_model.py) maps a normalised 60 × 8 window to six class probabilities in four stages: convolutional feature extraction, bidirectional recurrent modelling, temporal attention and classification (Figure 2). All convolutional, recurrent and dense layers use L2 weight regularisation (5×10^{-5}), and the model has 840,647 parameters.

Conv1D feature extraction. Three convolutional blocks extract local motion patterns, such as the acceleration peak of a footstep, and shorten the sequence. Block 1 has two Conv1D layers with 64 filters (kernel sizes 5 and 3; a kernel of five samples spans 250 ms), block 2 two layers with 128 filters (kernel size 3) and block 3 one layer with 256 filters (kernel size 3). Every convolution uses ReLU activation and batch normalisation, and every block ends with max pooling and dropout (0.15, 0.2 and 0.2). The sequence is reduced from 60 × 8 to 30 × 64, 15 × 128 and finally 7 × 256.

BiLSTM temporal modelling. A bidirectional LSTM with 128 units per direction reads the seven feature vectors forwards and backwards and returns the full sequence $h_1, \ldots, h_7$ (7 × 256), followed by dropout 0.25. Each h_t therefore describes one step of the window in the context of both the preceding and the following steps.

Temporal attention. A dense layer with 256 tanh units and a dense layer with one output assign a score e_t to every step, a softmax over the seven steps turns the scores into weights α_t, and each step is rescaled by its weight, where W, b, w and c are the weights and biases of the two dense layers:

$$e_t = w^T \tanh(W h_t + b) + c, \quad t = 1, \ldots, 7 \tag{4.1}$$

$$\alpha_t = \exp(e_t) / \Sigma_{k=1}^{7} \exp(e_k) \tag{4.2}$$

$$z_t = \alpha_t h_t \tag{4.3}$$

The weighted sequence $z_1, \ldots, z_7$ keeps its shape (7 × 256), so that the next layer can still use the temporal order.

Summary and classification. A second bidirectional LSTM with 64 units per direction summarises the weighted sequence into one 128-value vector (dropout 0.3). Dense layers with 128 and 64 ReLU units, each followed by batch normalisation and dropout (0.3 and 0.2), and a six-unit softmax output layer produce the class probabilities.

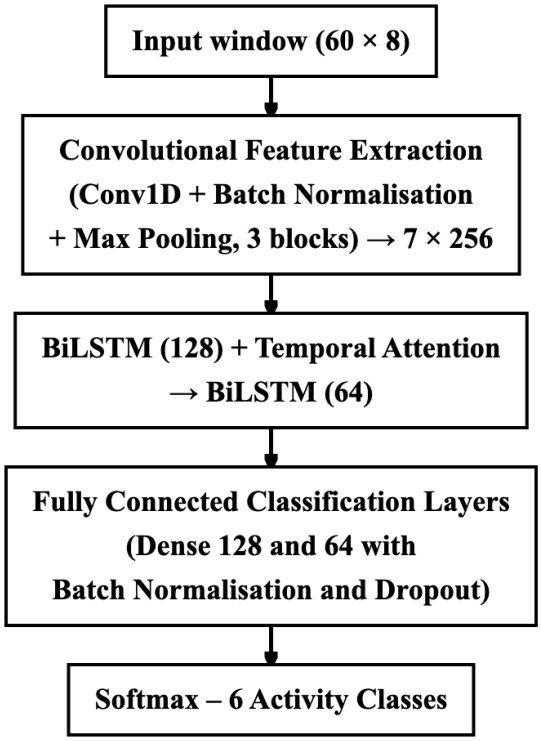


Figure 2: Conv1D–BiLSTM–Attention Model Architecture

4.8 Training Configuration

The model is trained with the following settings:

- **Loss function:** focal loss with $\gamma = 2$ and $\alpha = 0.25$. Focal loss multiplies the cross-entropy of each window by $(1 - p)^{\gamma}$, where p is the probability given to the correct class, so windows that are already classified confidently contribute little and training concentrates on difficult windows.
- **Class weights:** balanced class weights computed with scikit-learn (Section 4.5).
- **Optimiser:** Adam with an initial learning rate of 0.001; the learning rate is halved when the validation loss has not improved for five epochs (minimum 10^{-6}).
- **Batch size:** 64.
- **Early stopping:** training stops when the validation accuracy has not improved for 15 epochs, and the weights of the best epoch are restored.

The model was trained once with these settings for at most 60 epochs. Training stopped after 53 epochs and kept the weights of epoch 38, which had the highest validation accuracy (83.6 percent on the validation subjects).

4.9 Real-Time Prediction Pipeline

For each window received from the browser, the server (backend/app.py, backend/model_service.py) performs the steps given formally as Algorithm 3 and shown in Figure 4. The request must contain a 60 × 6 array of finite numbers (otherwise HTTP 400). The window is preprocessed and normalised (Section 4.6), and the six class probabilities are softened with a fixed temperature of 1.5 and averaged within the session with an exponential moving average (weight 0.4 for the newest window). If the summed per-axis variances of

both the raw accelerometer and the raw gyroscope are below 0.12, the label is Still with confidence 1.0. Otherwise, an averaged probability below 0.40 gives the label Uncertain, and the total motion variance limits the allowed classes: below 0.8 only Still and Hand Activity, below 2.0 also Stairs, below 5.0 also Walking and Jogging, and otherwise all classes; a disallowed class is replaced by the most probable allowed one. The final label is the most frequent of the session's last five labels, and it is stored if a session is active.

The response contains the final label, the network's own top class, the confidence, all probabilities and the name of any rule that changed the result. The temperature and rule thresholds were set by hand and were not tuned on the test data.

PROPOSED ALGORITHMS

This chapter gives the three main procedures of the system: data preparation and training (prepare_data_v2.py, backend/train_model_v2.py), the shared window preprocessing (backend/preprocessing.py) and real-time prediction (backend/app.py, backend/model_service.py).

5.1 Data Preparation and Model Training

Algorithm 1: Data Preparation and Model Training

Input: UCI HAR, WISDM and Heterogeneity recordings; labelled phone CSV files; class map M (Table 3)

Output: trained network f, fitted scaler S, label encoder E

W ← ∅

for each source s ∈ {UCI HAR, WISDM, Heterogeneity, phone} **do**

    convert acceleration to m/s² (UCI HAR: total acceleration × 9.80665) and angular velocity to rad/s

    align accelerometer and gyroscope readings by timestamp and resample to 20 Hz

    split the signal into continuous segments at every gap longer than 1 s

    cut each segment into 60-sample windows with step 30 and map their labels with M

    assign each window to the training, validation or test split by person (public data) or by recording (phone data), seed 42

    W ← W ∪ windows of s

end for

D_{train} ← training windows of W, at most 2,500 per class and dataset; phone windows are not capped

D_{val} ← validation windows of W, at most 600 per class and dataset

for each phone window x ∈ D_{train} **do**

    repeat 10 times: add a copy with random 3-D rotation, amplitude factor U(0.9, 1.1) and sensor noise to D_{train}

end for

X_{train} ← PREPROCESS(D_{train}); X_{val} ← PREPROCESS(D_{val}) *// Algorithm 2, 60 × 8 per window*

fit S on X_{train} only; compute balanced class weights from D_{train}

f ← CREATE_MODEL() *// Section 4.7*

train f on S(X_{train}) with focal loss ($\gamma = 2$, $\alpha = 0.25$), Adam (learning rate 10^{-3}) and batch size 64

stop when the accuracy on S(X_{val}) has not improved for 15 epochs and restore the best weights

return f, S, E

5.2 Window Preprocessing

Algorithm 2: Window Preprocessing (PREPROCESS)

Input: raw window x of 60 samples × 6 channels (accelerometer a in m/s², gyroscope g in rad/s)

Output: feature window of 60 samples × 8 channels

for each channel of x **do**
    apply a third-order Butterworth low-pass filter (5 Hz) forwards and backwards
end for
$g_a \leftarrow$ zero-phase Butterworth low-pass of the filtered accelerometer channels (0.3 Hz) *// gravity*
b ← filtered accelerometer channels − g_a *// body acceleration*
$m_b \leftarrow \|b\|$ and $m_g \leftarrow \|$filtered gyroscope channels$\|$ for every sample
return [b, filtered gyroscope channels, m_b, m_g]

5.3 Real-Time Prediction

Algorithm 3: Real-Time Prediction for One Window

Input: raw window x (60 × 6); network f; scaler S; session state: averaged probabilities $\bar{p}$ and label history H

Output: final label y, network class $\hat{y}$, confidence c, applied rule r, class probabilities p

if shape(x) ≠ (60, 6) **or** x contains a non-finite value **then return** HTTP 400 error
q ← f(S(PREPROCESS(x))) *// Algorithm 2*
p ← softmax(log q / T) with T = 1.5; $\hat{y} \leftarrow$ argmax p
if $\bar{p}$ is empty **then** $\bar{p} \leftarrow$ p **else** $\bar{p} \leftarrow 0.4\,p + 0.6\,\bar{p}$
y ← argmax $\bar{p}$; c ← max $\bar{p}$; r ← none
$v_a \leftarrow$ sum of per-axis variances of the raw accelerometer; $v_g \leftarrow$ same for the raw gyroscope
if $v_a < 0.12$ **and** $v_g < 0.12$ **then**
    y ← Still; c ← 1.0; r ← low-variance-still; reset $\bar{p}$ to Still
else
    if c < 0.40 **then** y ← Uncertain
    A ← classes allowed for the intensity $v_a + v_g$ (limits 0.8, 2.0 and 5.0, Section 4.9)
    if $y \notin A$ **and** y ≠ Uncertain **then** y ← most probable class of $\bar{p}$ in A; c ← $\bar{p}$ of y; r ← intensity-gate
end if
append y to H; y ← most frequent label among the last five entries of H
if the window belongs to a session **then** store (y, $\hat{y}$, c, r, p) in SQLite
return y, $\hat{y}$, c, r, p

PROPOSED FLOWCHARTS AND DIAGRAMS

6.1 System Architecture Diagram

Figure 3 shows the main components of the system and the data flow between them.

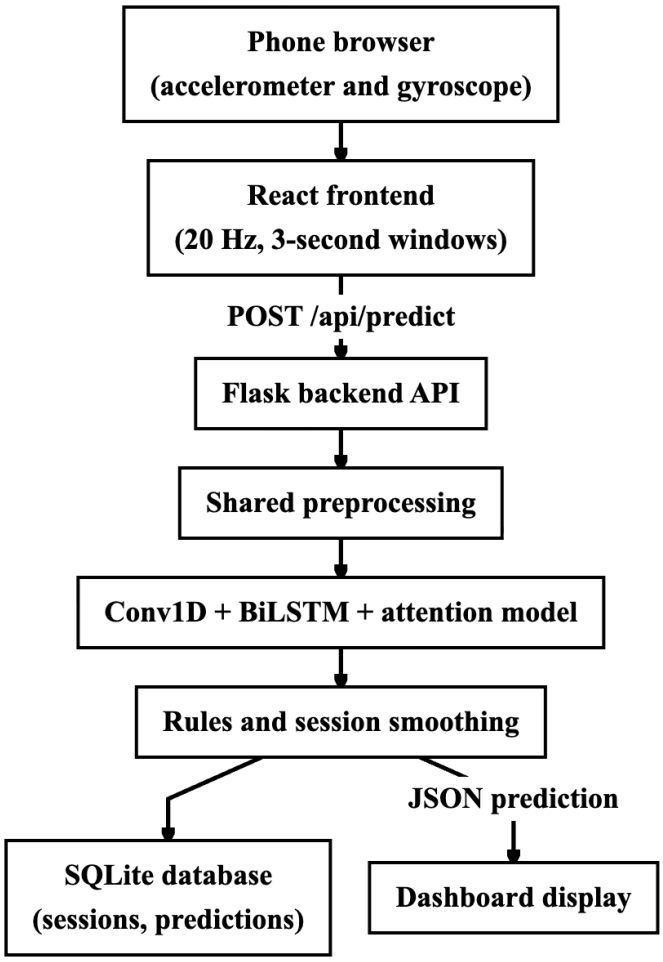


Figure 3: High-Level System Architecture

6.2 Prediction Pipeline Flowchart

Figure 4 shows the main steps taken for every window received from the browser (Section 4.9).

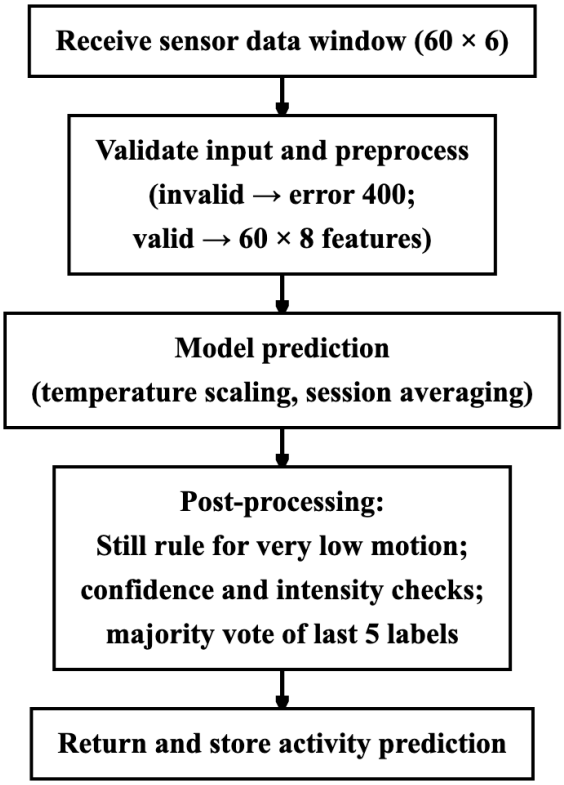


Figure 4: Prediction Pipeline Flowchart

IMPLEMENTATION (TOOLS AND TECHNOLOGY USED)

7.1 Overview of Technology Stack

The system consists of a React frontend, a Flask backend, a TensorFlow/Keras model and a local SQLite database. Table 5 lists the main tools; the versions are those installed on the development laptop (Apple M3, 16 GB RAM, macOS).

Table 5: Tools and Technologies Used

Layer	Technology	Version	Purpose in this project
Frontend	React, Vite, Chart.js	19.3.1, 8.3.5, 4.6.1	Browser client, development server with HTTPS, live charts
Backend framework	Flask, Flask-CORS	3.1.3, 6.0.5	REST API
Deep learning	TensorFlow, Keras	2.21.0, 3.15.1	Training and running the model
Signal processing	SciPy	1.17.1	Butterworth filters, resampling
Data processing	NumPy, pandas	2.4.6, 3.1.0	Arrays and CSV parsing
Machine learning utilities	scikit-learn	1.8.0	StandardScaler, LabelEncoder, metrics (same version that saved the scaler and encoder)
Storage and export	SQLite (Python standard library), fpdf2	built in, 2.8.9	Sessions, predictions and PDF reports
Testing	pytest	9.1.1	Automated backend tests
Languages	Python 3.11, JavaScript (ES2022)	—	Backend and machine learning; frontend

The Python packages are listed in backend/requirements.txt, requirements-train.txt (training scripts) and requirements-cloud.txt (optional module), and the frontend packages in frontend/package.json.

7.2 Frontend–Backend Communication

The browser client exists to stream sensor data to the model and display its output. The hook useSensorStream.js reads accelerationIncludingGravity (m/s²) and rotationRate (converted from degrees per second to rad/s) from the DeviceMotion interface, which mobile browsers expose only to HTTPS pages and, on iOS, only after the user grants permission. A 50 ms timer appends the latest reading as one sample, so the live path samples at a fixed 20 Hz, whereas uploaded CSV files are resampled by timestamp on the server. The client keeps the last 60 samples and posts a window to /api/predict after the first 60 samples and then every 30 samples, with at most three requests in flight. The returned label, confidence, network class and class probabilities are shown on a live view with an activity timeline; other views analyse uploaded CSV files, list stored sessions with CSV and PDF export, and show the

served model's settings and evaluation report. Stored recordings can be replayed through the same loop for demonstrations, clearly marked as not live. The Vite development server forwards /api requests to Flask (port 5001), so a phone on the same network needs only one address.

7.3 Backend Implementation

At start-up (create_app in backend/app.py) the model service loads the Keras network, label encoder, activity mapping and fitted scaler from the model folder and refuses to start unless they are consistent: the encoder classes must equal the activity names, the network must accept 60 × 8 input and output six classes, and the scaler must have eight features. backend/preprocessing.py provides the shared preprocess_window function (Algorithm 2), window validation and a CSV parser for the Android sensor-logger and dashboard formats. /api/predict implements Algorithm 3 with smoothing state kept per session in memory and protected by a lock, and /api/predict/csv applies the same pipeline to every window of an uploaded file. Further endpoints provide health and model information, replay recordings, session management and CSV and PDF exports. Sessions and predictions are stored in two SQLite tables, sessions and predictions; each prediction row holds the window time, final activity, network class, confidence, applied rule and all class probabilities.

The evaluation scripts backend/evaluate.py and backend/compare_models.py run the model through the application's own code on the phone recordings and the held-out windows and produce all results of Chapter 8 (evaluation_report.json, comparison.json). The 25 pytest tests in backend/tests/ cover the API with a stub model (valid and invalid input, the stationary rule, sessions, CSV formats, gaps and exports) and check the preprocessing and the real model files (consistency, probabilities summing to one, correct classification of the Still and Jogging recordings); all 25 passed on 29 October 2026.

RESULT DISCUSSION AND ANALYSIS

8.1 Evaluation Design and Metrics

All results in this chapter were produced by the project's evaluation scripts (backend/evaluate.py and backend/compare_models.py), which use the same preprocessing and inference code as the live application. The metrics on the public held-out subjects were additionally recomputed from the saved test windows and the trained model files for this report, and they agree exactly with the saved evaluation output. The test subjects and recordings were fixed before training, and no setting was tuned on them. Each result belongs to one of the test sets in Table 6.

Table 6: Test Sets Used in the Evaluation

Test set	Content	Used for training?	Results in
Public held-out subjects	27,352 windows from 10 WISDM subjects, 2 Heterogeneity users and the 9 UCI HAR test subjects	No	Tables 7 and 8; Figure 5
Phone Stairs session	117 windows: two files of one stair-climbing session recorded on two devices	No	Table 9
Held-out phone walking recording	48 windows	No	Table 9
Phone recordings seen in training	230 windows: Still, Jogging and one Walking recording	Yes	Table 9

Accuracy is the fraction of correctly classified windows. For each class, precision is TP / (TP + FP) and recall TP / (TP + FN), where TP, FP and FN are the true positives, false positives and false negatives, and the F1-score is their harmonic mean; macro values are unweighted averages over the classes present. For a phone recording, all windows share one true class, so its accuracy equals the recall of that class. "Model output" is the network's top class; "dashboard pipeline" is the label after the rules and smoothing of Section 4.9, which is what a user sees.

8.2 Results on Held-Out Public-Dataset Subjects

Table 7 gives the per-class and overall results of the model output on the 27,352 windows of the 21 held-out subjects, and Figure 5 shows the confusion matrix as percentages of each true class. The diagonal of the confusion matrix contains 21,308 correctly classified windows, which gives the accuracy of 0.779.

Table 7: Per-Class and Overall Results on Held-Out Public-Dataset Subjects (model output)

Class	Precision	Recall	F1-score	Windows (support)
Hand Activity	0.707	0.473	0.567	5,908
Jogging	0.888	0.886	0.887	1,181
Sports	0.798	0.772	0.785	4,893
Stairs	0.846	0.928	0.885	4,571
Still	0.711	0.895	0.793	7,340
Walking	0.889	0.833	0.860	3,459
Macro average	**0.807**	**0.798**	**0.796**	27,352
Accuracy			**0.779** (21,308 of 27,352)	27,352

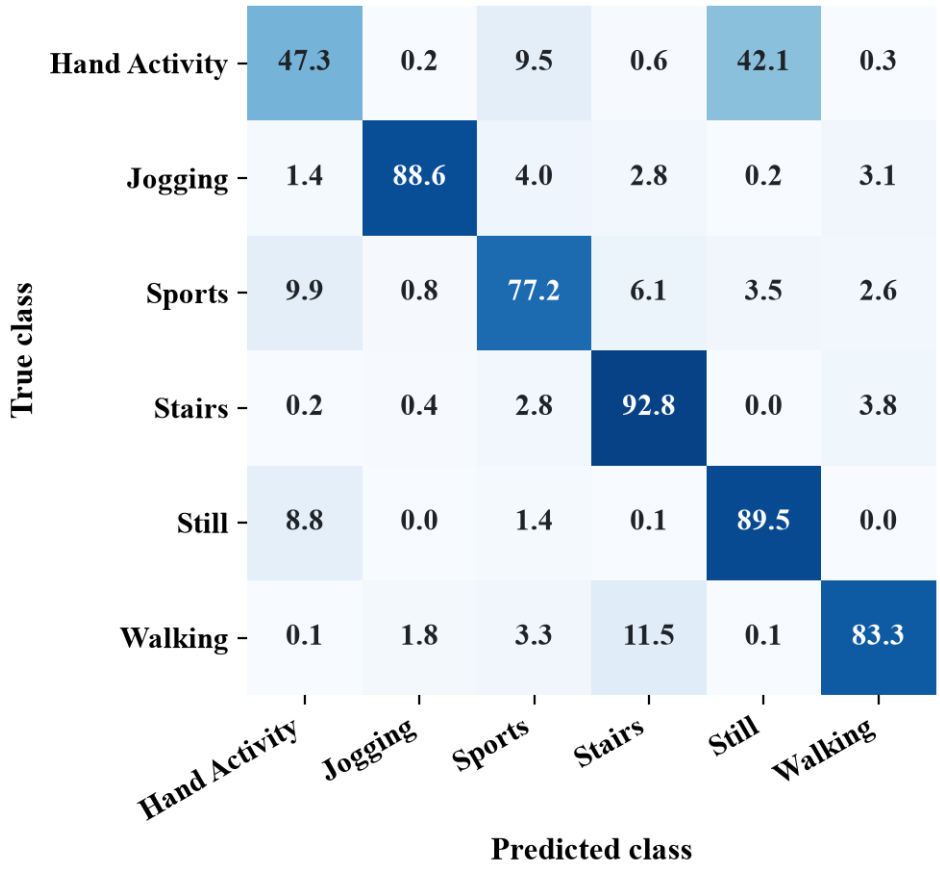


Figure 5: Confusion Matrix on Held-Out Public-Dataset Subjects (percentage of each true class)

Table 8: Results per Source Dataset (held-out subjects, model output)

Source	Held-out subjects	Windows	Accuracy	Macro F1-score (classes present)	Stairs recall
WISDM	10	15,357	63.5%	0.671	0.757
Heterogeneity	2	9,656	96.0%	0.964	0.985
UCI HAR	9	2,339	97.9%	0.979	0.994

The results can be interpreted as follows:

1. **Stairs is recognised on data from new people.** Of the 4,571 true Stairs windows, 4,240 are classified correctly (recall 0.928). The model predicts Stairs for 5,013 windows in total, of which these 4,240 are correct (precision 0.846); the remaining 773 Stairs predictions are mainly Walking windows (398) and Sports windows (298). Stairs recall is 0.757 on the WISDM subjects, 0.985 on the Heterogeneity users and 0.994 on the UCI HAR subjects (Table 8).
2. **Hand Activity is the weakest class.** Only 2,794 of the 5,908 Hand Activity windows are found (recall 0.473), and 2,489 of them are predicted as Still. Hand Activity comes only from WISDM, where the phone was in the pocket while the hands typed, wrote or folded clothes, so the phone records little more than a standing or sitting person (Section 2.3). The confusion in the other direction (649 Still windows predicted as Hand Activity) lowers the precision of both classes and is the main reason why the accuracy on the WISDM subjects (63.5 percent) is lower than on the other two datasets.
3. **The overall accuracy is below the target.** The accuracy of 77.9 percent is below the 80 percent objective of Section 3.2. Excluding the WISDM subjects, the accuracy exceeds 96 percent, so the shortfall is concentrated in the Hand Activity–Still confusion.

8.3 Results on Phone Recordings

Table 9 gives the results on the labelled phone recordings, both for the model output and for the full dashboard pipeline.

Table 9: Results on Phone Recordings (accuracy of model output / of dashboard pipeline)

Recording	True class	Used for training	Windows	Model output	Dashboard pipeline
Stairs session, device 1	Stairs	No	63	61.9% (39)	84.1% (53)
Stairs session, device 2	Stairs	No	54	27.8% (15)	16.7% (9)
Held-out walking recording	Walking	No	48	87.5% (42)	81.3% (39)
Still recording	Still	Yes	103	100%	100%
Jogging recording	Jogging	Yes	64	100%	100%
Training walking recording	Walking	Yes	63	100%	100%

On the stair-climbing session, 54 of the 117 windows are classified as Stairs (model output, recall 46.2 percent); the others are predicted as Walking (30), Sports (17), Hand Activity (6), Still (6) and Jogging (4). The two files differ strongly: the first, which probably comes from the same phone as one of the Walking training recordings, reaches 84.1 percent (53 of 63) after the dashboard pipeline, while the second, from another device, reaches only 16.7 percent (9 of 54). Over all 165 held-out phone windows, the model output is correct for 58.2 percent (96) and the dashboard pipeline for 61.2 percent (101). Recordings used for training are classified correctly, which only confirms that the pipeline reproduces training behaviour.

These results rest on one stair-climbing session from two devices and one walking recording, which is far too little to establish performance on phones in general. Twelve live phone-browser sessions recorded on 27 and 28 October 2026 (about 6.4 minutes, 229 predictions) confirm that live capture, prediction and storage work end to end, but the activities performed were not recorded, so they cannot measure accuracy.

8.4 Comparison with Published Approaches

Table 10 lists results reported in the literature reviewed in Chapter 2 next to the results of this project.

Table 10: Comparison with Published Approaches

Approach	Feature extraction	Classes	Reported result	Test data
SVM [1]	561 handcrafted features	6	96% accuracy	UCI HAR test set
MLP [2]	43 handcrafted features	6	91.7% accuracy	WISDM (2010 collection)
LSTM [24]	Learned	6	92.1% accuracy	WISDM
CNN-LSTM [14]	Learned	6	92% accuracy	UCI HAR
LSTM-CNN [15]	Learned	6	95.78% accuracy	UCI HAR
DeepConvLSTM [6]	Learned	18	0.915 weighted F1-score (not accuracy)	Opportunity gestures, including the Null class
This project	Learned	6	77.9% accuracy, macro F1-score 0.796; 54 of 117 phone Stairs windows correct	Held-out subjects of WISDM, Heterogeneity and UCI HAR; one phone Stairs session

These figures are not directly comparable: the published results come from single benchmark datasets with their own classes and protocols, whereas this project uses a mixture of three datasets with six merged classes and person-level held-out subjects. Even the 97.9 percent on the UCI HAR test subjects uses three merged classes instead of the six original ones and cannot be compared with [1], [14] or [15]. The table therefore shows where the project stands, not that its model is better than published methods.

8.5 Limitations

1. **Very little labelled phone test data.** The only held-out phone Stairs data is one 95-second session on two devices with strongly differing results, and there are no phone recordings of Hand Activity or Sports, so real-world performance is not established.
2. **Live accuracy not measured.** Live sessions were recorded without a record of the activities performed, and iPhone behaviour was not tested.
3. **Sampling, device and position.** The browser takes the latest reading every 50 ms, whereas the training data were resampled by timestamp; the phone position in the Heterogeneity dataset and the device models of the project recordings are unknown, and iOS and Android may report axes with different signs.
4. **Accuracy.** The accuracy of 77.9 percent on held-out subjects is below the 80 percent target, Hand Activity is often confused with Still, and four windows of the held-out Walking recording are predicted as Stairs.
5. **Untuned post-processing, no ablation and limited classes.** The temperature and rule thresholds were set by hand, the effect of the attention layer was not measured, upstairs and downstairs are merged, and the optional sign-in and report features were not tested.

CONCLUSION AND FUTURE SCOPE

9.1 Summary of the Work

This project developed a system that recognises six activities from a smartphone's accelerometer and gyroscope. A data pipeline converts the WISDM, UCI HAR and Heterogeneity datasets and the project's phone recordings to the units, sampling rate and window format of the live input and splits them by person and recording before augmentation. A Conv1D–BiLSTM–attention model trained on these data is served by a Flask API that applies the training preprocessing, post-processing rules and per-session smoothing to 20 Hz windows streamed from a phone browser, and stores the predictions in SQLite. On 27,352 windows from 21 held-out public-dataset subjects, the model reaches an accuracy of 77.9 percent, a macro precision of 0.807, a macro recall of 0.798 and a macro F1-score of 0.796, with a Stairs recall of 0.928; on the held-out phone stair session, 54 of 117 windows are classified as Stairs. Evaluation scripts regenerate every reported number, and 25 automated tests check the API, preprocessing and model files.

9.2 Conclusion

The project shows that a phone browser, a small server and a hybrid convolutional and recurrent network suffice for a working real-time activity recognition system, and that data preparation is central to it: training sources must be converted to the exact format of the live input, and the evaluation must keep people and recordings separate from training, because overlapping windows and augmented copies otherwise make the measured accuracy unreliable. With these measures, locomotion classes such as Stairs, Walking and Jogging are recognised well on held-out people, while Hand Activity remains difficult for a phone in the pocket.

Measured against the objectives of Section 3.2, the data pipeline (objective 2), the real-time inference service (objective 4) and the dashboard with storage and export (objective 5) were achieved. The leakage-free evaluation (objective 3) was achieved for the public-dataset subjects, although the phone test data are very limited. The classifier (objective 1) works and is served by the application, but its accuracy of 77.9 percent on held-out subjects is below the 80 percent target.

Stair recognition on phones is promising but not established. The single held-out phone session gives very different results on its two devices, and the classes Hand Activity and Sports could not be tested on phone data at all. The most important next step is therefore to collect new labelled phone recordings.

9.3 Future Scope

1. **New labelled phone data.** Several people should record stair climbing up and down, walking, Hand Activity and Sports with the phone in the pocket and in the hand, with some people kept as a test set never used for training; the client's raw CSV export can be used for this.

2. **Labelled live tests.** Live sessions should be recorded together with the activities performed, so that the accuracy of the complete live system, including on iPhones, can be measured.
3. **Tuning and ablation.** The temperature and rule thresholds could be chosen on the validation subjects, and training without the attention layer or with a single LSTM layer would show which components help.
4. **On-device inference.** Converting the model to TensorFlow Lite would remove network delay and keep sensor data on the phone.
5. **Additional sensors and classes.** A barometer could separate stair climbing from walking through height changes, and separate up and down classes could be added once suitable data exist.

REFERENCES

[1] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *Proc. 21st European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning (ESANN)*, Bruges, Belgium, 2013, pp. 437–442.

[2] J. R. Kwapisz, G. M. Weiss, and S. A. Moore, "Activity recognition using cell phone accelerometers," *ACM SIGKDD Explorations Newsletter*, vol. 12, no. 2, pp. 74–82, Dec. 2010, doi: 10.1145/1964897.1964918.

[3] J. Yang, M. N. Nguyen, P. P. San, X. L. Li, and S. Krishnaswamy, "Deep convolutional neural networks on multichannel time series for human activity recognition," in *Proc. 24th Int. Joint Conf. Artificial Intelligence (IJCAI)*, Buenos Aires, Argentina, 2015, pp. 3995–4001.

[4] M. Zeng, L. T. Nguyen, B. Yu, O. J. Mengshoel, J. Zhu, P. Wu, and J. Zhang, "Convolutional neural networks for human activity recognition using mobile sensors," in *Proc. 6th Int. Conf. Mobile Computing, Applications and Services (MobiCASE)*, Austin, TX, USA, 2014, pp. 197–205, doi: 10.4108/icst.mobicase.2014.257786.

[5] N. Y. Hammerla, S. Halloran, and T. Plötz, "Deep, convolutional, and recurrent models for human activity recognition using wearables," in *Proc. 25th Int. Joint Conf. Artificial Intelligence (IJCAI)*, New York, NY, USA, 2016, pp. 1533–1540.

[6] F. J. Ordóñez and D. Roggen, "Deep convolutional and LSTM recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, Art. no. 115, Jan. 2016, doi: 10.3390/s16010115.

[7] A. Stisen, H. Blunck, S. Bhattacharya, T. S. Prentow, M. B. Kjærgaard, A. Dey, T. Sonne, and M. M. Jensen, "Smart devices are different: Assessing and mitigating mobile sensing heterogeneities for activity recognition," in *Proc. 13th ACM Conf. Embedded Networked Sensor Systems (SenSys)*, Seoul, South Korea, 2015, pp. 127–140, doi: 10.1145/2809695.2809718.

[8] G. M. Weiss, "WISDM smartphone and smartwatch activity and biometrics dataset," *UCI Machine Learning Repository*, 2019, doi: 10.24432/C5HK59. [Online]. Available: **https://archive.ics.uci.edu/dataset/507/**

[9] H. Blunck and S. Bhattacharya, "Heterogeneity activity recognition," *UCI Machine Learning Repository*, 2015, doi: 10.24432/C5689X. [Online]. Available: **https://archive.ics.uci.edu/dataset/344/**

[10] N. Y. Hammerla and T. Plötz, "Let's (not) stick together: Pairwise similarity biases cross-validation in activity recognition," in *Proc. ACM Int. Joint Conf. Pervasive and Ubiquitous Computing (UbiComp)*, 2015, pp. 1041–1051, doi: 10.1145/2750858.2807551.

[11] J. L. Reyes-Ortiz, L. Oneto, A. Samà, X. Parra, and D. Anguita, "Transition-aware human activity recognition using smartphones," *Neurocomputing*, vol. 171, pp. 754–767, Jan. 2016, doi: 10.1016/j.neucom.2015.07.085.

[12] V. S. Murahari and T. Plötz, "On attention models for human activity recognition," in *Proc. ACM Int. Symp. Wearable Computers (ISWC)*, 2018, pp. 100–103, doi: 10.1145/3267242.3267287.

[13] Y. Zhao, R. Yang, G. Chevalier, X. Xu, and Z. Zhang, "Deep residual Bidir-LSTM for human activity recognition using wearable sensors," *Mathematical Problems in Engineering*, vol. 2018, Art. no. 7316954, 2018, doi: 10.1155/2018/7316954. Preprint: arXiv:1708.08989.

[14] R. Mutegeki and D. S. Han, "A CNN-LSTM approach to human activity recognition," in *Proc. Int. Conf. Artificial Intelligence in Information and Communication (ICAIIC)*, Fukuoka, Japan, 2020, pp. 362–366, doi: 10.1109/ICAIIC48513.2020.9065078.

[15] K. Xia, J. Huang, and H. Wang, "LSTM-CNN architecture for human activity recognition," *IEEE Access*, vol. 8, pp. 56855–56866, 2020, doi: 10.1109/ACCESS.2020.2982225.

[16] G. M. Weiss, K. Yoneda, and T. Hayajneh, "Smartphone and smartwatch-based biometrics using activities of daily living," *IEEE Access*, vol. 7, pp. 133190–133202, 2019, doi: 10.1109/ACCESS.2019.2940729.

[17] B. Oluwalade, S. Neela, J. Wawira, T. Adejumo, and S. Purkayastha, "Human activity recognition using deep learning models on smartphones and smartwatches sensor data," in *Proc. 14th Int. Joint Conf. Biomedical Engineering Systems and Technologies (BIOSTEC)*, vol. 5: HEALTHINF, 2021, pp. 645–650, doi: 10.5220/0010325906450650.

[18] S. Alam, T. Zhang, T. Feng, H. Shen, Z. Cao, D. Zhao, J. Ko, K. Somasundaram, S. S. Narayanan, S. Avestimehr, and M. Zhang, "FedAIoT: A federated learning benchmark for artificial intelligence of things," arXiv:2310.00109, 2023.

[19] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998, doi: 10.1109/5.726791.

[20] S. Ha and S. Choi, "Convolutional neural networks for human activity recognition using multiple accelerometer and gyroscope sensors," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, Vancouver, BC, Canada, 2016, pp. 381–388, doi: 10.1109/IJCNN.2016.7727224.

[21] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[22] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997, doi: 10.1109/78.650093.

[23] Y. Guan and T. Plötz, "Ensembles of deep LSTM learners for activity recognition using wearables," *Proc. ACM Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 1, no. 2, Art. no. 11, Jun. 2017, doi: 10.1145/3090076.

[24] Y. Chen, K. Zhong, J. Zhang, Q. Sun, and X. Zhao, "LSTM networks for mobile human activity recognition," in *Proc. Int. Conf. Artificial Intelligence: Technologies and Applications (ICAITA)*, Bangkok, Thailand, 2016, pp. 50–53, doi: 10.2991/icaita-16.2016.13.

[25] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015, arXiv:1409.0473.

[26] T. T. Um, F. M. J. Pfister, D. Pichler, S. Endo, M. Lang, S. Hirche, U. Fietzek, and D. Kulić, "Data augmentation of wearable sensor data for Parkinson's disease monitoring using convolutional neural networks," in *Proc. 19th ACM Int. Conf. Multimodal Interaction (ICMI)*, Glasgow, UK, 2017, pp. 216–220, doi: 10.1145/3136755.3136817.

LIST OF PUBLICATIONS

None at the time of submission.

PLAGIARISM REPORT

[Attach the plagiarism (similarity) report generated with the institute's checking tool here before submission.]